

OS Project Description (2569)

Concurrent Reservation System using C/C++ Message Queue

วัตถุประสงค์

Java ไม่ได้

ให้นักศึกษาพัฒนาระบบ **Concurrent Reservation System** ด้วยภาษา C หรือ C++ บน Linux/Docker โดยใช้ **Message Queue** สำหรับการสื่อสารระหว่าง Client และ Server และใช้กลไก **Concurrency Control** เพื่อป้องกัน Race Condition ที่เกิดจาก Worker หลายตัวเข้าถึงข้อมูลการจองร่วมกัน

โครงงานนี้ครอบคลุมแนวคิดหลัก ได้แก่

- Interprocess Communication ด้วย Message Queue
- Client-Server Architecture
- Concurrent Processing
- Process หรือ Thread
- Shared Resource
- Race Condition และ Critical Section
- Mutex หรือ Semaphore
- การทดลองระบบบน Docker ผ่านหลาย Terminal

1. ลักษณะของระบบ - โจทย์

ให้พัฒนาระบบจองทรัพยากร โดยเลือก **หนึ่งสถานการณ์** จาก การจองที่นั่งในโรงภาพยนตร์ การจองที่นั่งบนเครื่องบิน การจองห้องประชุม หรือ การจองบัตรเข้าร่วมกิจกรรม เช่น คอนเสิร์ต

กำหนดให้ระบบมีทรัพยากรที่สามารถจองได้อย่างน้อย **20 รายการ** เช่น หมายเลขที่นั่ง 1-20 ห้องประชุม 1-20 หรือหมายเลขบัตร 1-20 Client หลายตัวสามารถส่งคำขอไปยัง Server พร้อมกันได้

คำสั่งขั้นต่ำประกอบด้วย **UI like text**

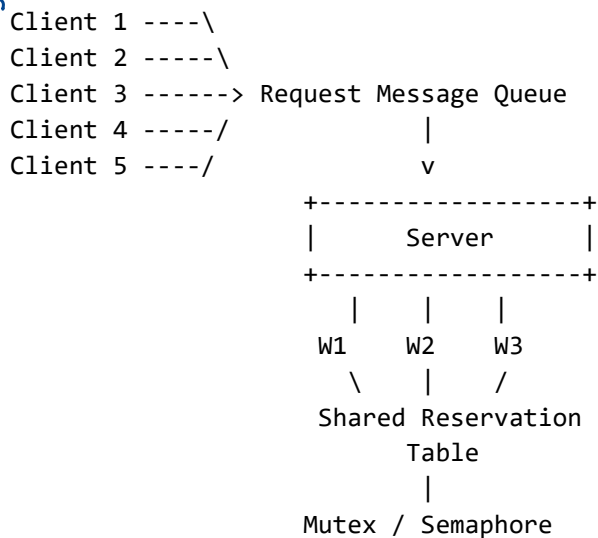
```
LIST
STATUS <resource_id>
RESERVE <resource_id>
CANCEL <resource_id>
QUIT
```

ตัวอย่างเช่น

```
RESERVE 10
STATUS 10
CANCEL 10
```

2. สถาปัตยกรรม

Client *ลัทธิก็ได้* ระบบต้องใช้ Message Queue เป็นช่องทางรับคำขอจาก Client



- Server ต้องมี Worker อย่างน้อย **3 ตัว** เพื่อประมวลผลคำขอหลายรายการพร้อมกัน
- Message Queue ใช้ได้ทั้ง System V Message Queue หรือ POSIX Message Queue
- Server รับข้อความจาก Request Queue แล้วส่งให้ Worker ประมวลผล แล้วส่งผลลัพธ์กลับ

2.1 Shared Resource

สงวนใจ

Server ต้องมีข้อมูลส่วนกลาง ที่ Worker หลายตัวสามารถเข้าถึงพร้อมกันได้ เช่น

Resource	Status	Owner
1	AVAILABLE	-
2	RESERVED	Client-3
3	AVAILABLE	-
4	RESERVED	Client-1

Hint: นักศึกษาต้องระบุให้ชัดเจนว่า

- ตัวแปรหรือข้อมูลใดเป็น Shared Data
- Critical Section อยู่บริเวณใด
- ใช้ Mutex หรือ Semaphore ควบคุมบริเวณใด
- เหตุใด Message Queue เพียงอย่างเดียวจึงไม่เพียงพอในการป้องกัน Race Condition

3. Race Condition Experiment (*important*)

ให้ Client อย่างน้อย 5 ตัวส่งคำสั่งเพื่อจองทรัพยากรเดียวกัน เช่น **RESERVE 10** ในช่วงเวลาใกล้เคียงกัน

เมื่อ Server มี Worker หลายตัว แต่ยังไม่มีการ Synchronization อาจเกิดสถานการณ์

```

Worker 1: check Resource 10 → AVAILABLE
Worker 2: check Resource 10 → AVAILABLE
Worker 3: check Resource 10 → AVAILABLE
  
```

ก่อนที่จะ Worker ใดจะเขียนผลกลับ

เพื่อให้ Race Condition เกิดได้ชัดเจน ให้นักศึกษาใส่ random delay ระหว่างขั้นตอน check และ update

ตัวอย่างแนวคิด

```
if (reservation[id] == AVAILABLE) {  
    random_delay(); // อาจใช้ delay ประมาณ 50-500 ms แบบสุ่ม  
    reservation[id] = client_id;  
}
```

Random delay นี้ใช้เพื่อ ขยาย Race Window สำหรับการทดลองเท่านั้น ไม่ใช่ส่วนหนึ่งของระบบจริง

3.1 การแก้ Race Condition

หลังจากแสดงให้เห็นปัญหาแล้ว ให้ปรับโปรแกรมโดยใช้ Mutex หรือ Semaphore

แม้ยังมี random delay อยู่ ระบบที่ Synchronize ถูกต้องต้องให้ผลว่า

```
Client 1 : FAILED  
Client 2 : FAILED  
Client 3 : SUCCESS  
Client 4 : FAILED  
Client 5 : FAILED
```

และต้องมีผู้จองทรัพยากรหมายเลข 10 สำเร็จเพียง **หนึ่ง Client เท่านั้น**

3.2 การทดลองที่ต้องทำ

นักศึกษาต้องทดลองอย่างน้อย 3 กรณี

Experiment 1 — Sequential Baseline

- ใช้ Worker เพียง 1 ตัว
- ตรวจสอบว่าระบบทำงานถูกต้องและไม่มี Race Condition แม้มี Client หลายตัวส่งข้อความเข้าคิว

Experiment 2 — Concurrent **without** Synchronization

- ใช้ Worker ^{เพิ่ม Worker} อย่างน้อย 3 ตัว
- ไม่ใช้ Mutex/Semaphore และใส่ random delay ระหว่าง check และ update
- แสดงหลักฐานว่ามีโอกาสเกิด Race Condition ^{ถ้าใส่อันนี้แล้วจะกลับปกติ} ← ต้องทำโน้ต

Experiment 3 — Concurrent **with** Synchronization

- ใช้ Worker เท่าเดิมและ random delay เท่าเดิม
- เพิ่ม Mutex หรือ Semaphore
- แสดงว่าทรัพยากรหนึ่งรายการไม่สามารถถูกจองพร้อมกันโดย Client มากกว่าหนึ่งรายการ

4. Server Log ← เพื่อบันทึก

Server ต้องแสดง Log ที่ช่วยให้เห็นการทำงานพร้อมกัน เช่น

```
[Worker-1] received RESERVE 10 from Client-1  
[Worker-2] received RESERVE 10 from Client-2
```

```
[Worker-1] check Resource 10: AVAILABLE  
[Worker-2] check Resource 10: AVAILABLE
```

```
[Worker-1] entering critical section  
[Worker-1] Resource 10 reserved by Client-1
```

[Worker-1] leaving critical section

[Worker-2] entering critical section

[Worker-2] Resource 10 already reserved

[Worker-2] leaving critical section

Log ควรแสดงอย่างน้อย Client ID, Worker ID, Command, Resource ID, เวลาโดยประมาณหรือ sequence number, การเข้าและออก Critical Section

5. Docker Environment

โปรแกรมทั้งหมดต้อง Compile และทำงานได้บน Docker

ตัวอย่างการทดลอง

Terminal 1 : Server

Terminal 2 : Client 1

Terminal 3 : Client 2

Terminal 4 : Client 3

Terminal 5 : Client 4

Terminal 6 : Client 5

จะได้ทบทวนตัวกันได้อีก

สามารถใช้ Container เดียวแล้วเปิดหลาย Terminal ผ่าน `docker exec -it <container-name> bash`

หรือออกแบบหลาย Container ได้ตามความเหมาะสม

6. ข้อกำหนดขั้นต่ำ

ระบบต้องมีคุณสมบัติต่อไปนี้

1. เขียนด้วย C หรือ C++
2. ใช้ System V หรือ POSIX Message Queue
3. เลือกสถานการณ์ระบบจองหนึ่งแบบ ได้แก่ โรงภาพยนตร์ เครื่องบิน ห้องประชุม หรือกิจกรรม/คอนเสิร์ต
4. มีทรัพยากรที่สามารถจองได้อย่างน้อย 20 รายการ
5. มี Client อย่างน้อย 5 ตัว
6. มี Worker อย่างน้อย 3 ตัว
7. มี Shared Reservation Data
8. รองรับคำสั่ง LIST, STATUS, RESERVE, CANCEL
9. สามารถสร้าง Race Condition ได้โดยตั้งใจ
10. ใช้ random delay ระหว่าง check และ update
11. ใช้ Mutex หรือ Semaphore แก้ Race Condition
12. แสดง Log ของ Worker และ Critical Section
13. Compile และ Run ผ่าน Docker ได้
14. ทดลองจากหลาย Terminal

7. สิ่งที่ต้องส่ง

Source Code

อย่างน้อยประกอบด้วย

1. ^{ตัวโค้ด}
 - server.c / server.cpp
 - client.c / client.cpp
 - Dockerfile
2. README.md ^{ต้องให้ทบทวน} ต้องระบุ:
 - วิธี Build Docker Image
 - วิธี Run Container
 - วิธีเปิด Server
 - วิธีเปิด Client หลายตัว
 - รูปแบบ Message Queue ที่ใช้
 - คำสั่งที่ Client ร้องรับ
 - วิธีทดลอง Race Condition
 - วิธีเปิด/ปิด Synchronization

3. 8. รายงาน

รายงานประมาณ 6-10 หน้า ประกอบด้วย

1. ชื่อโครงการ รายชื่อและ ID ของสมาชิก
2. Project Description แบบกระชับ (5-10 บรรทัด)
3. System Architecture : ^{รูปภาพส่วประกอบอะไรบ้าง} ^{ฟังก์ชัน 2. → เขียนในสไลด์ใน canvas} ^{e.g. ch 2 pg. 47}
4. Message Queue Design → ^{แนบในโน้ต}
5. Message Structure
6. Server Concurrency Model
7. Shared Resource และ Critical Section
8. สาเหตุของ Race Condition
9. บทบาทของ random delay ในการทดลอง
10. Synchronization Mechanism ที่เลือกใช้
11. ผลการทดลองทั้ง 3 กรณี
12. ข้อจำกัดและแนวทางปรับปรุงหรือพัฒนาต่อ

รายงานใช้ font **Th Saraban New 14pt** เป็น Normal และใช้ Style: Heading 1, Heading 2, etc. ให้เหมาะสม

9. การสาธิต

ระหว่างการ Demo ต้องแสดงอย่างน้อย

- Demo 1 Client หลายตัวส่งคำสั่งต่างกันพร้อมกัน

Test : เติมหลอดเข้าไป
→ เครื่องที่ใช้รองรับหลอดได้มากแค่ไหน

- Demo 2 Client หลายตัวพยายามจองทรัพยากรเดียวกัน โดย **ไม่ใช่** Synchronization ต้องแสดง Race Condition ให้เห็นอย่างน้อยหนึ่งครั้ง
- Demo 3 ทำการทดลองเดิมอีกครั้ง**หลังเพิ่ม** Mutex หรือ Semaphore ต้องแสดงว่ามี Client เพียงหนึ่งตัวจองทรัพยากรได้สำเร็จ

10. เกณฑ์การประเมิน

หัวข้อ	คะแนน
Message Queue และ IPC	15
Concurrent Server / Worker Design	15
Critical Section and Concurrency Control	10
Race Condition Experiment	15
Docker และ Multi-Terminal Demonstration	10
Code Quality และ Error Handling	5
รายงานและการวิเคราะห์	15
การสาธิตและตอบคำถาม	15
รวม	100